Distributed Optimization for Machine Learning

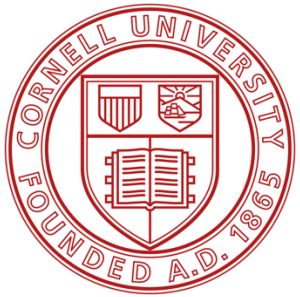
Lecture 21 – Analog computing for energy-efficient AI: Part I

ECE 5290/7290 & ORIE 5290

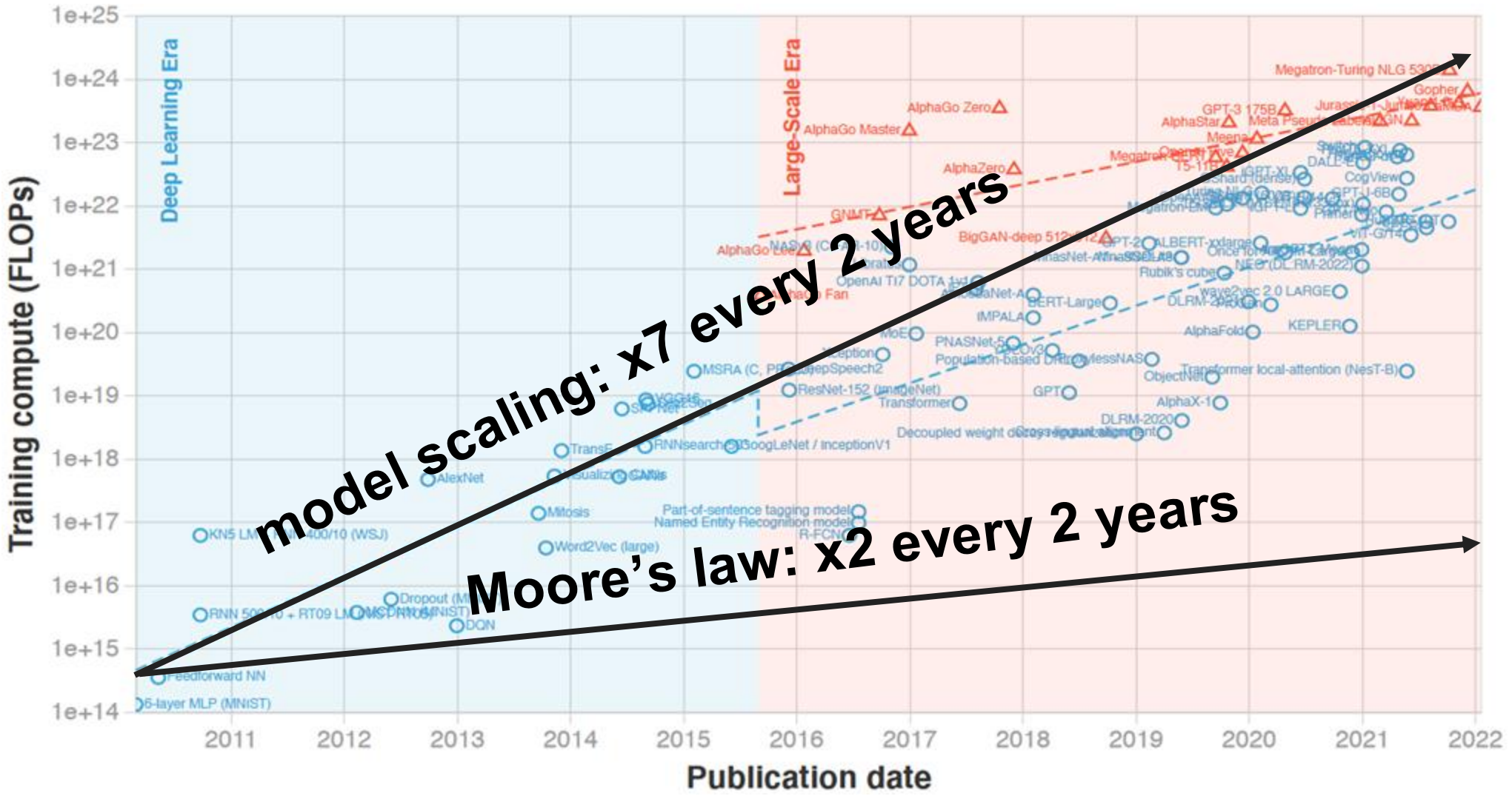
Tianyi Chen

School of Electrical and Computer Engineering
Cornell Tech, Cornell University

November 17, 2025

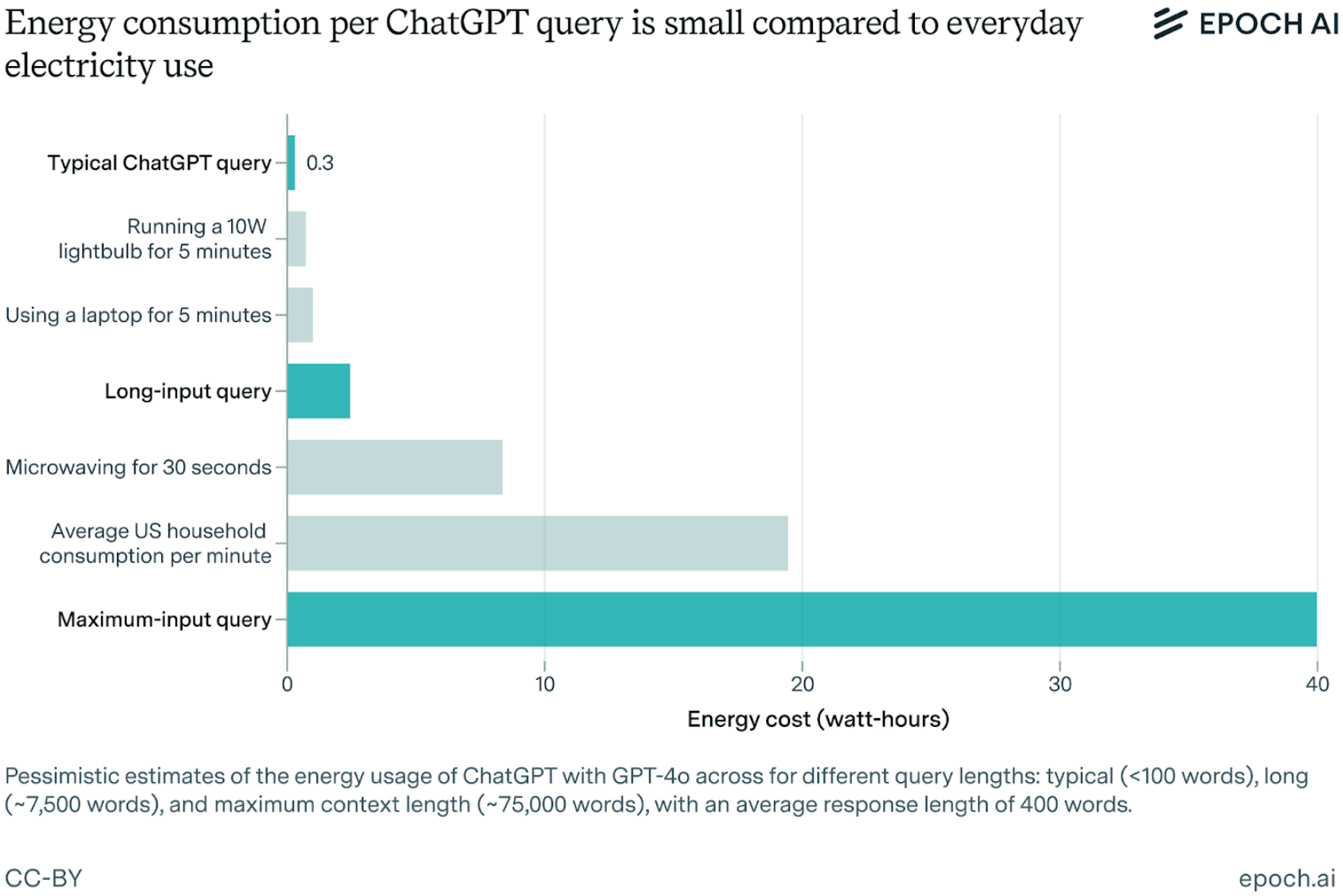


Training large models requires large compute



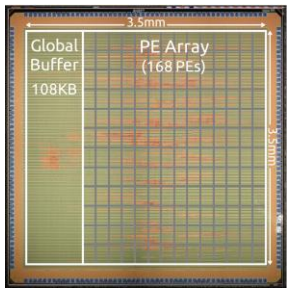
Sevilla et al., "Compute trends across three eras of machine learning," **IJCNN 2022.**

The carbon footprint of using ChatGPT



Source: Josh You at Epoch AI (How much energy does ChatGPT use?)

Current computing hardware



CPU **GPU** **FPGA** **ASIC**

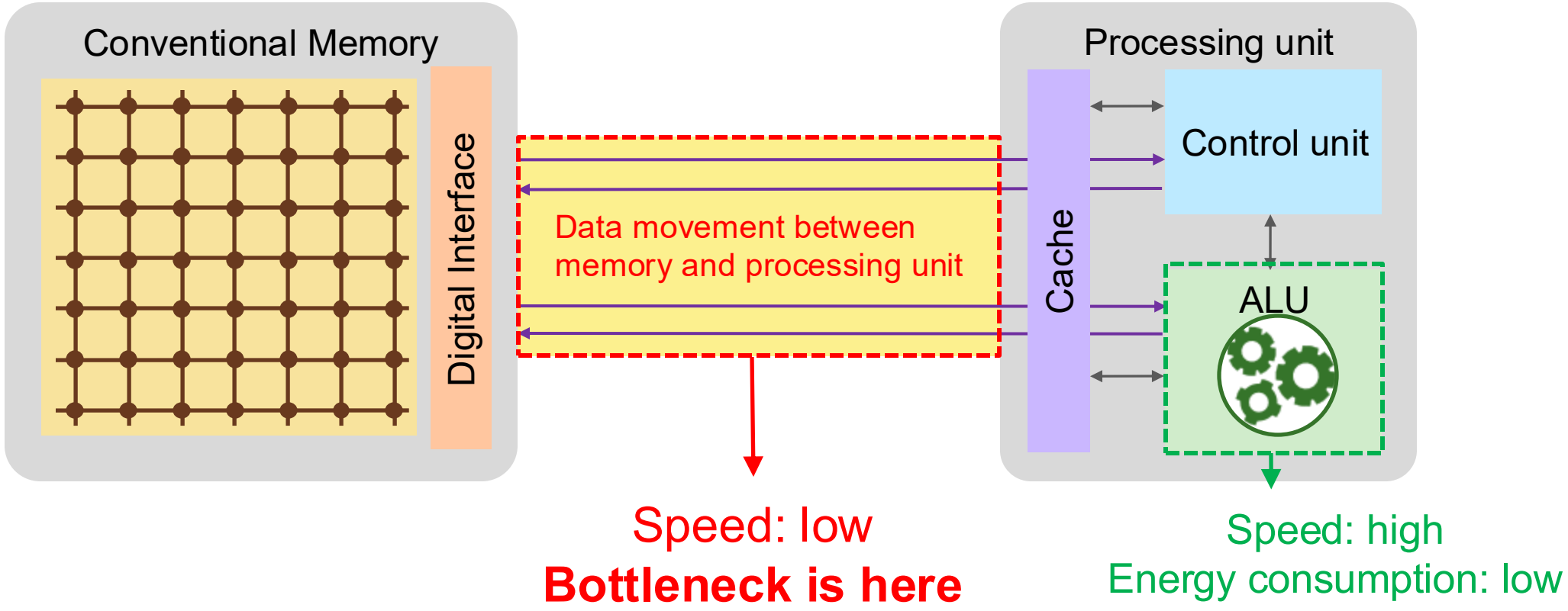
Flexibility **Efficiency**

They all belong to **out-memory computation architecture**

https://www.intel.com/content/www/us/en/newsroom/news/12th-gen-intel-core-i9-12900ks-worlds-fastest-desktop-processor.html#gs.9yghxm
https://www.nvidia.com/en-us/data-center/a100/
https://www.xilinx.com/products/boards-and-kits/1-t5lnks.html

Bottleneck: Data movement is costly

Out-memory architecture

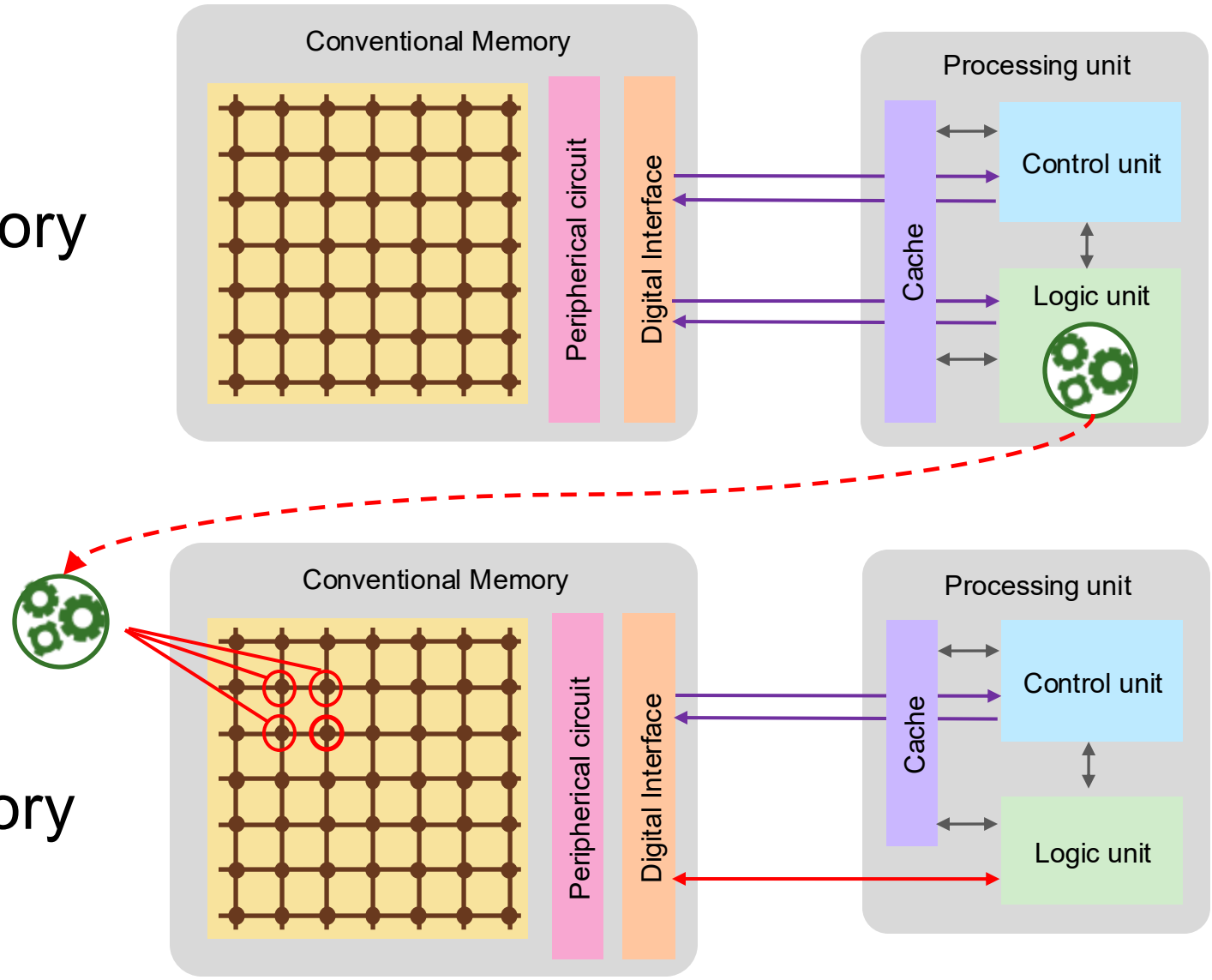


Out-memory vs in-memory computation

Out-memory

In-memory computation can be done via either **digital** or **analog** devices

In-memory



Data flow in deep models

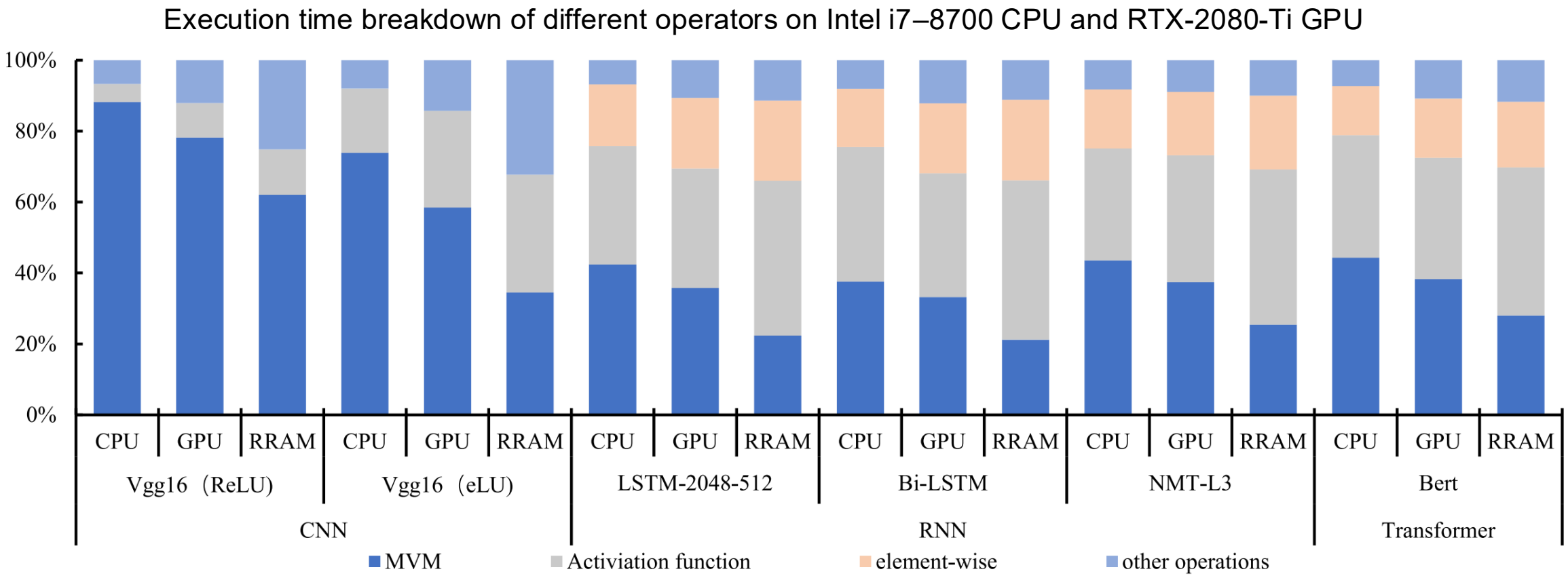
Linear operation
(Large matrix)

Non-linear activation
(Small vector)

Fix computationally intensive layers in memory!

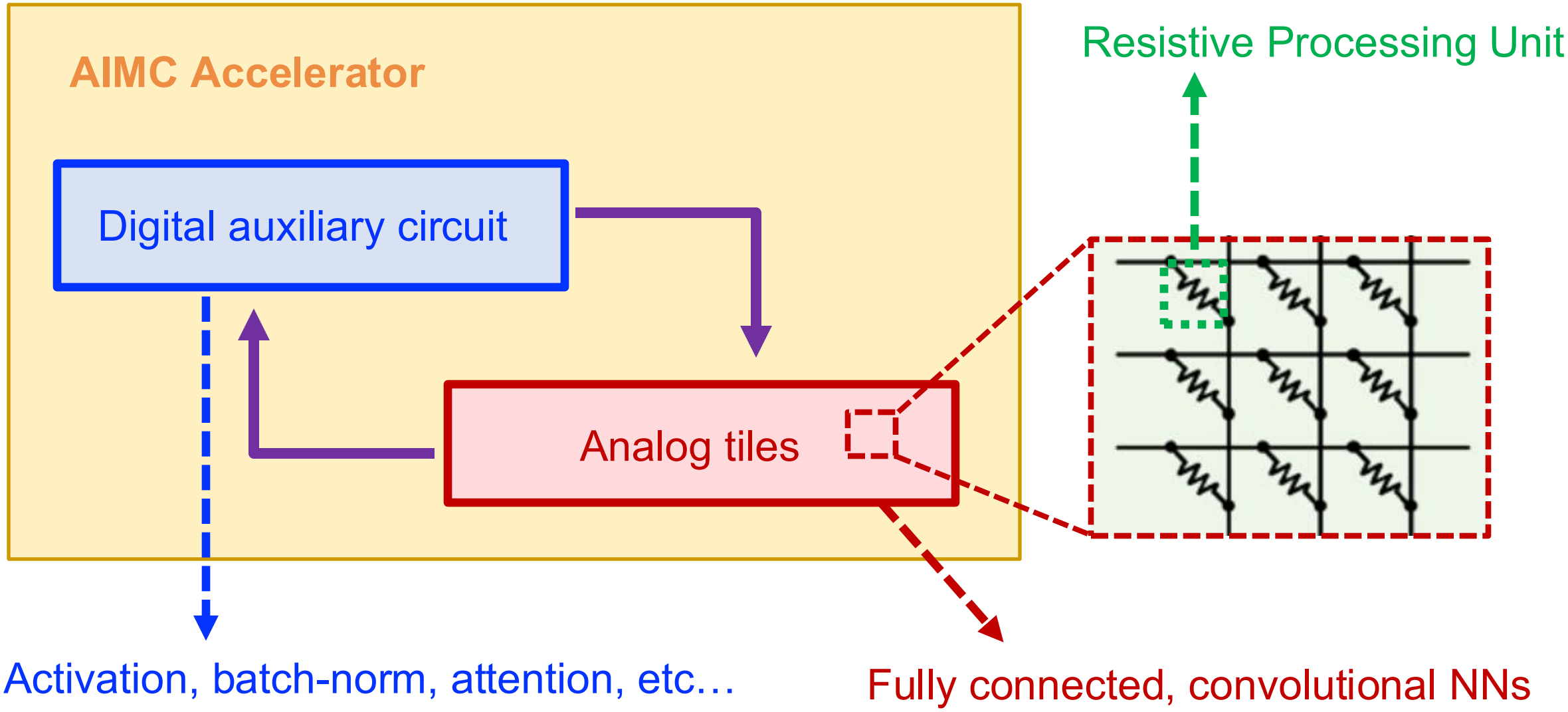
Motivation of analog in-memory computation (AIMC)

Matrix-Vector Multiplication (MVM) is the bottleneck of the DNN



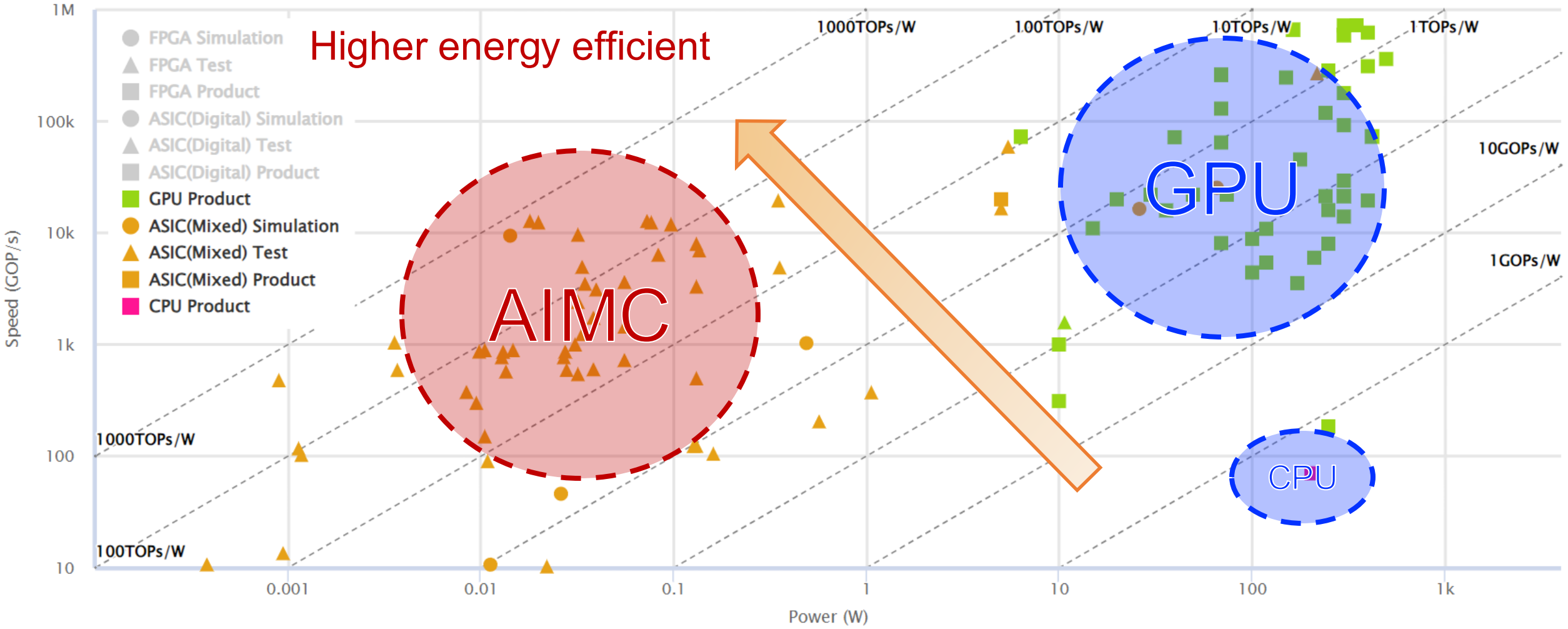
Z. Zhang, et al. "A universal RRAM-based DNN accelerator with programmable crossbars beyond MVM operator." **TCAD 41.7 (2021): 2094-2106**.

Overview of AIMC hardware



A Sebastian, et al. “Memory devices and applications for in-memory computing”. **Nature nanotechnology, 2020, 15(7): 529-544**
S Ambrogio, et al. “Equivalent-accuracy accelerated neural-network training using analogue memory”. **Nature, 2018, 558(7708): 60-67**.

Analog in-memory computing accelerates inference!



AIMC hardware: 10x-100x energy-efficient than GPU

K Guo, et al. "Neural Network Accelerator Comparison" **[Online]**. Available: https://nicsefc.ee.tsinghua.edu.cn/projects/neural-network-accelerator/

Significant real impact on inference cost

GPT-4o

GPT-4o is our most adv
Turbo with stronger vis
knowledge cutoff.

Learn about GPT-4o ↗

Model

gpt-4o

gpt-4o-2024-05-13

Vision pricing calculator

Set width Se

150 px by

Low resolution

← Post

William Wang
@WilliamWangNLP

Just another day, waking up to find out that the undergrad hired by your grad student generated a $20,000 OpenAI API bill in the last three days, without your authorization. 😅

10:42 AM · Jun 4, 2024 · **642.2K** Views

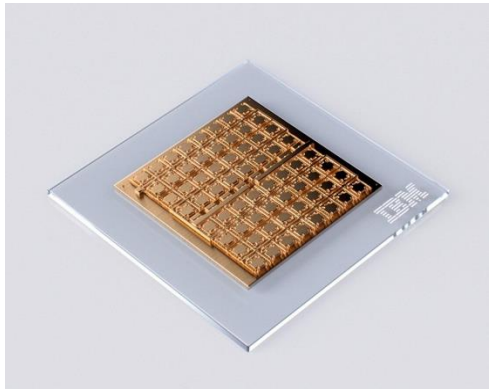
78 155 2.7K 398

Post your reply

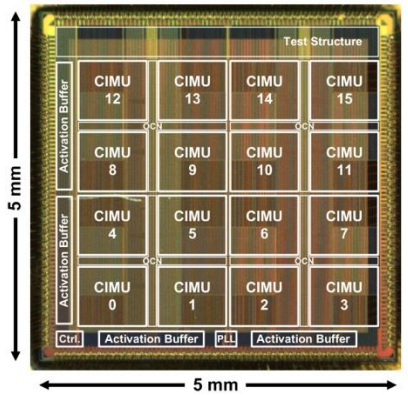
Reply

Images from Yin, Shukang, et al. "A survey on multimodal large language models." arXiv preprint arXiv:2306.13549 (2023), NVIDIA and https://medium.com/@ashwathkumarshettyr/multi-modal-large-language-models-1-introduction-f5e9bffc07df.

Recent AIMC chips



IBM HERMES chip
10.5 TOPS/W



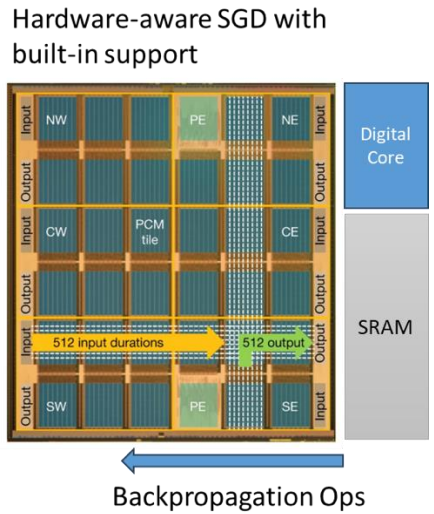
Enlarge AI chip
30 TOPS/W

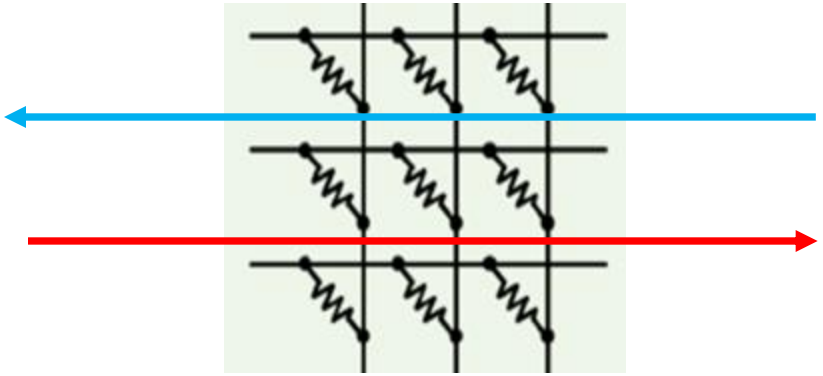
Mythic Analog Processor
6.3-8.3 TOPS/W

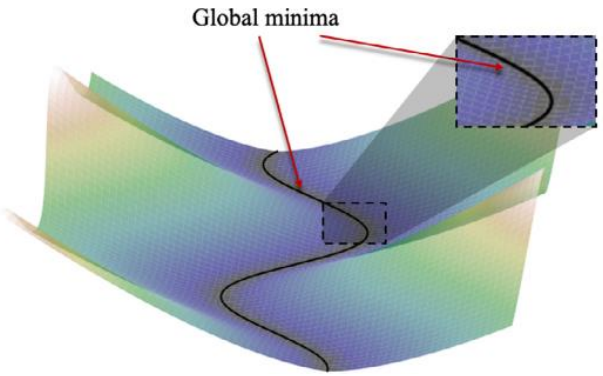
* TOPS/W = Tera Floating-point arithmetic Operations Per Second per Watt

M Le Gallo, et al. “A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference”**. Nature Electronics 6.9 (2023): 680-693.**
H Jia, et al. "15.1 a programmable neural-network inference accelerator based on scalable in-memory computing." IEEE International Solid-State Circuits Conference. Vol. 64, 2021.
M1076 Analog Matrix Processor. **https://mythic.ai/products/m1076-analog-matrix-processor/**

Scope of this lecture







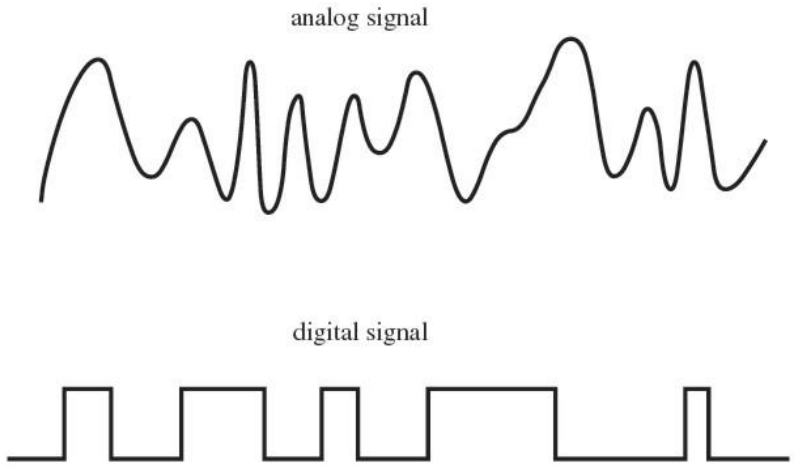
AIMC Circuit **Hardware property** **Analog AI**

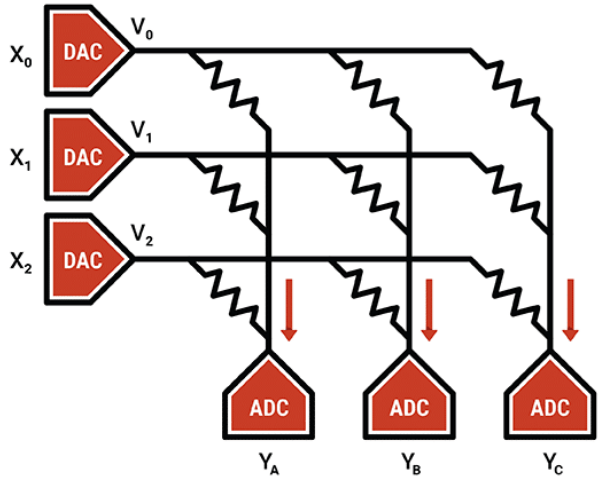
Hardware design

Algorithm design

Outline

- Background of Analog In-memory Computing
- Analog Inference: Hardware-aware Retraining





Resistive processing unit

Resistive processing unit (RPU): non-volatile electrical data storage unit

- Map the conductance (reciprocal of resistance) into the weight

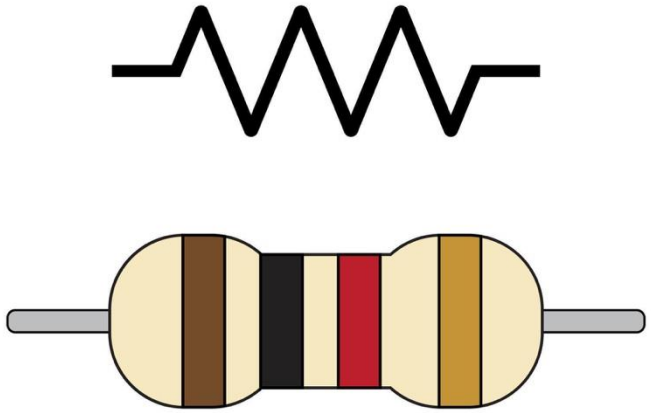
- The weight in RPU is an analog quantity (continuous quantity)

Analog Representation

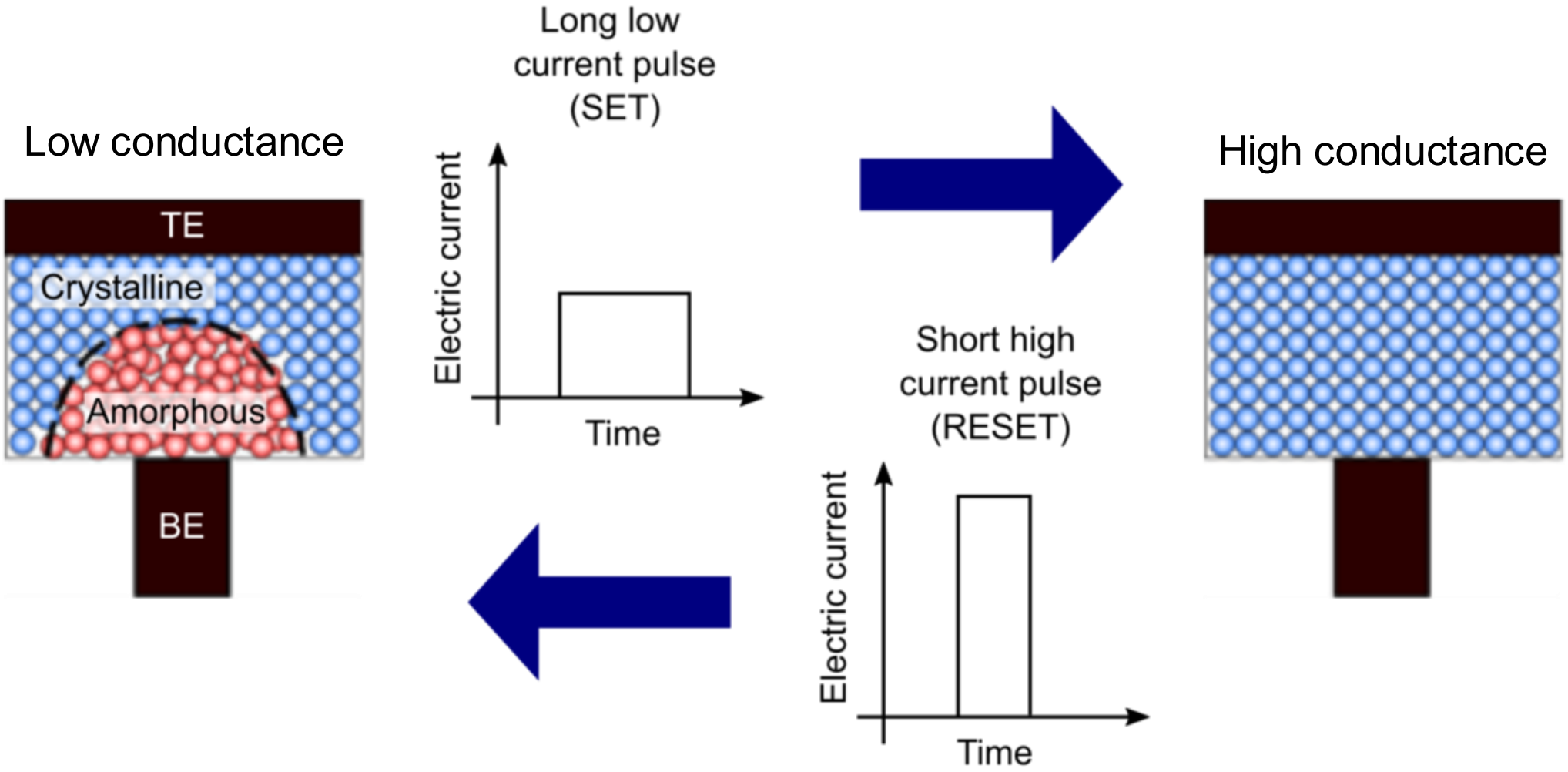
Conductance (nS)		Weight
10	->	0.1
20	->	0.2
30	->	0.3
40	->	0.4

Digital Representation

Code		Weight
001	->	0.1
010	->	0.2
011	->	0.3
100	->	0.4



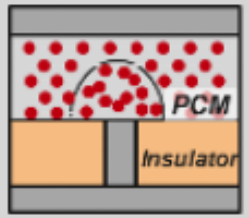
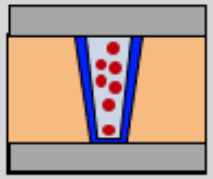
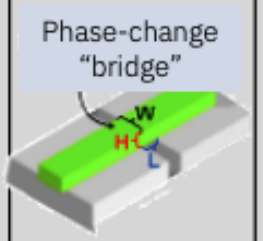
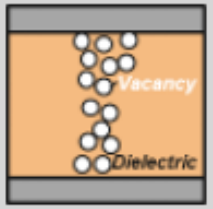
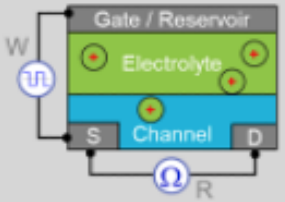
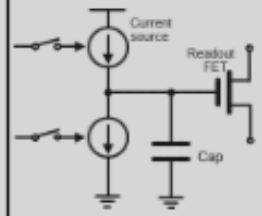
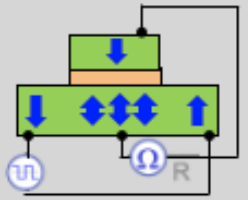
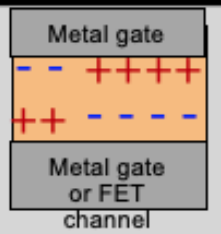
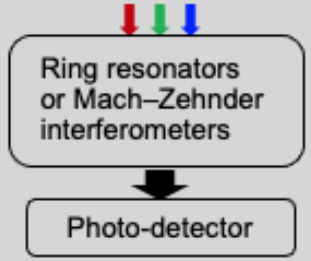
RPU example: Phase change memory



RPU can programmed by current pulses

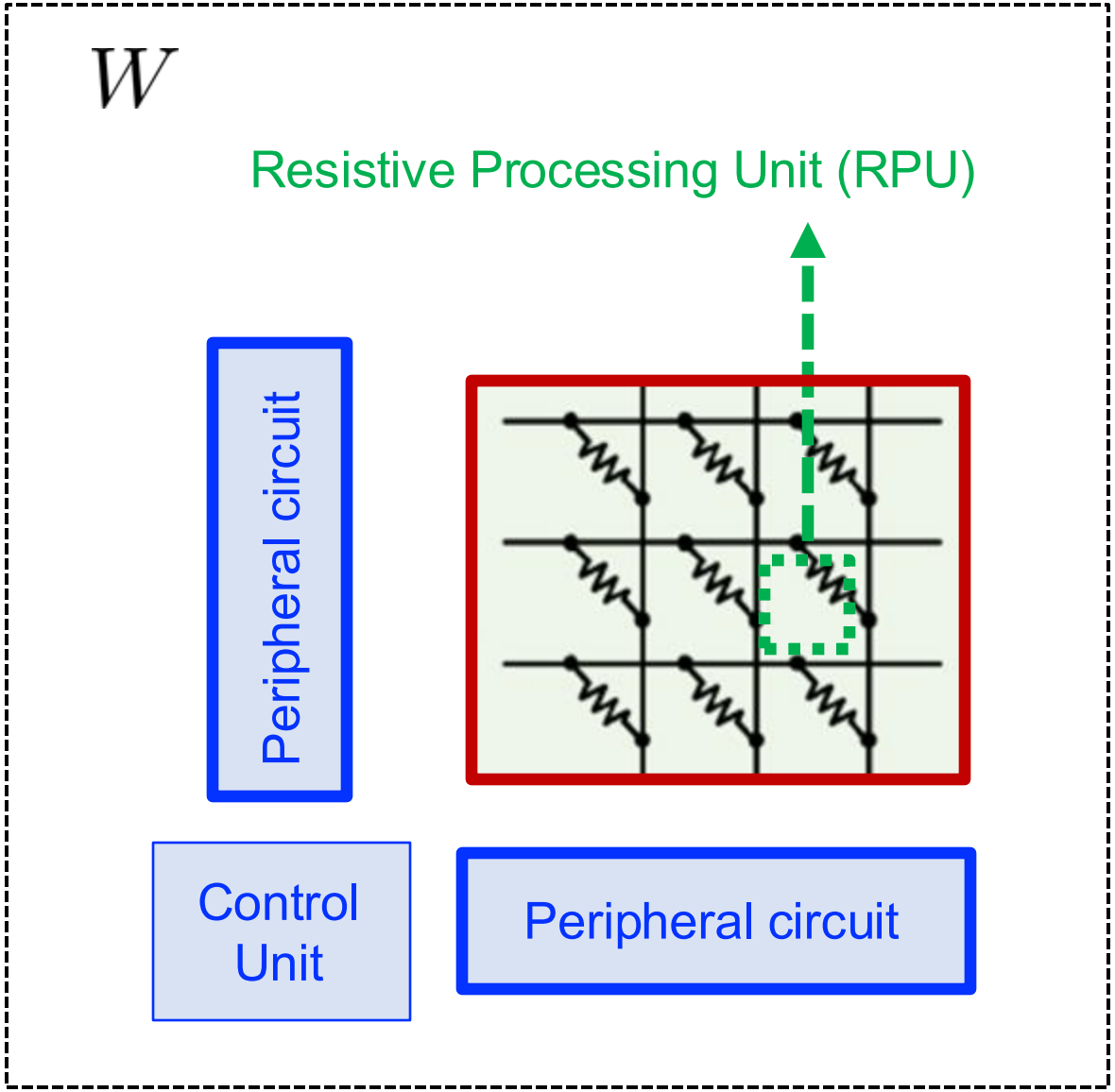
M Le Gallo, & A Sebastian. “An overview of phase-change memory device physics ”. **Journal of Physics D: Applied Physics, 53(21), 213002. 2020**

RPU candidates

Phase change memory (PCM) (crystalline/amorphous transition)			RRAM (filamentary)	ECRAM (Electrochem)	CMOS (capacitor)	MRAM & Ferroelectric (domain wall motion)		Photonics (transmission)
 mushroom	 confined surfactant					 magnetic	 FE	
+ Mature among storage-class memory			+ Bi-directional + Small	+ Symmetric + Low power	+Symmetric + Mature	+ Symmetric + Low power		+ Low power + Mature
- Abrupt reset - Asymmetry *(1-cell)* - Drift			- Stochastic - Asymmetry	- Open circuit voltage - Ion motion	- Cell size - Retention	- Domain wall control		- Cell size - Scalability (# of neurons)

Crossbar tile

- Arrange RPUs into a tile
- Peripheral circuits process the IO (analog-digital conversion)
- One crossbar tile stores one matrix W



Circuit class 101: Ohm's law and Kirchhoff law

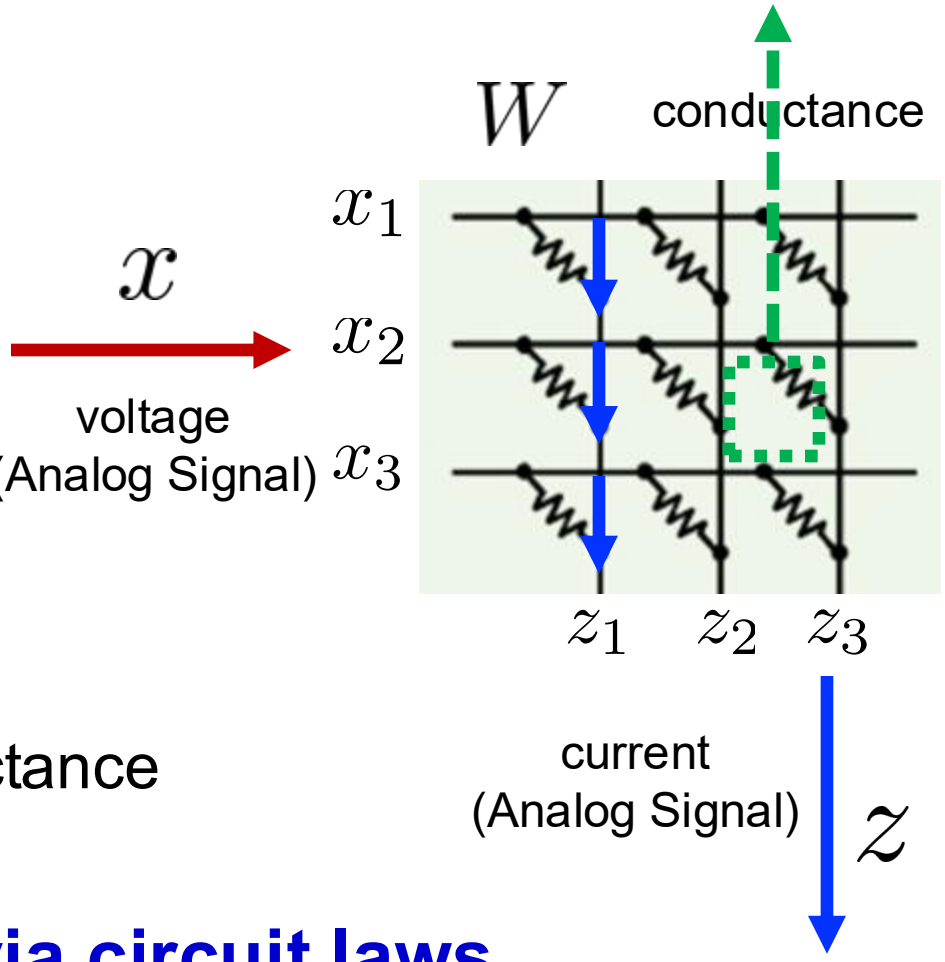
conductance

$$z = w \cdot x$$

current voltage

$$z_i = \underbrace{\sum_j \overbrace{W_{ij}x_j}^{\text{Ohm's law}}}_{\text{Kirchhoff's current law}}$$

Resistive Processing Unit (RPU)



x

voltage
(Analog Signal)

current
(Analog Signal)

z

- Basic circuit law: Ohm's and Kirchhoff's laws
- Data vector ***x*** is represented by the voltage
- Weight matrix ***W*** is represented by RPU's conductance

Matrix-vector multiplication done via circuit laws

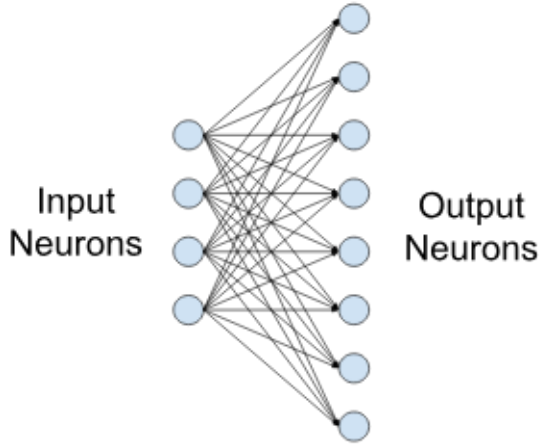
A Sebastian, et al. "Memory devices and applications for in-memory computing". **Nature nanotechnology, 2020, 15(7): 529-544**
S Ambrogio, et al. "Equivalent-accuracy accelerated neural-network training using analogue memory". **Nature, 2018, 558(7708): 60-67**.

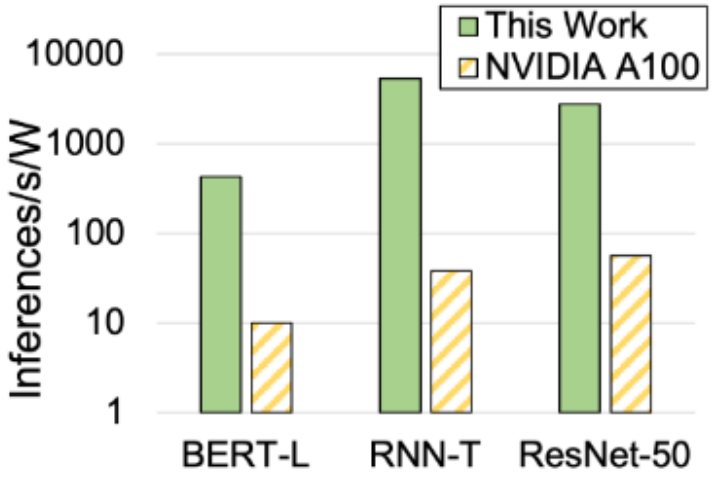
Linear layers on RPU array

- Linear layer is implemented by directly computing

  Output = Weight x Input + Bias

- Save heavy multiply–accumulate (MAC) computation





	Digital	Analog
MAC	$O(N^2)$	O(N)

Compare with NVIDIA A100:

- 100x-300x MVM energy-efficiency
- 40x-140x overall energy-efficiency

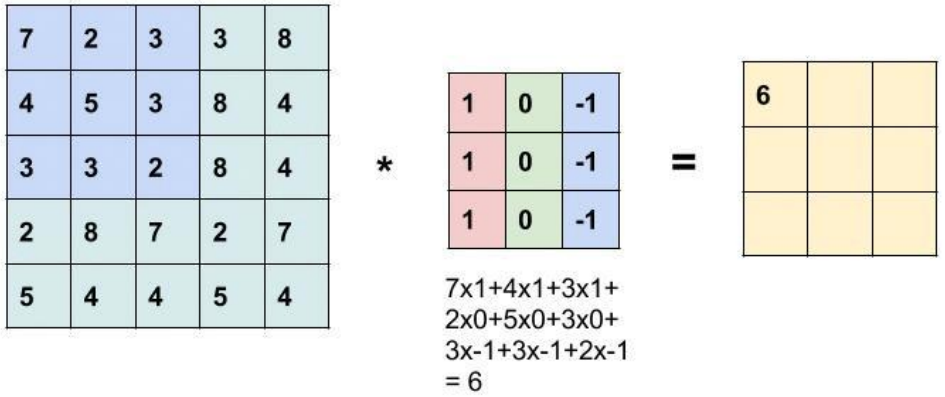
S Jain, et al. "A heterogeneous and programmable compute-in-memory accelerator architecture for analog-AI using dense 2-D mesh." **TVLSI 31.1 (2022): 114-127.**

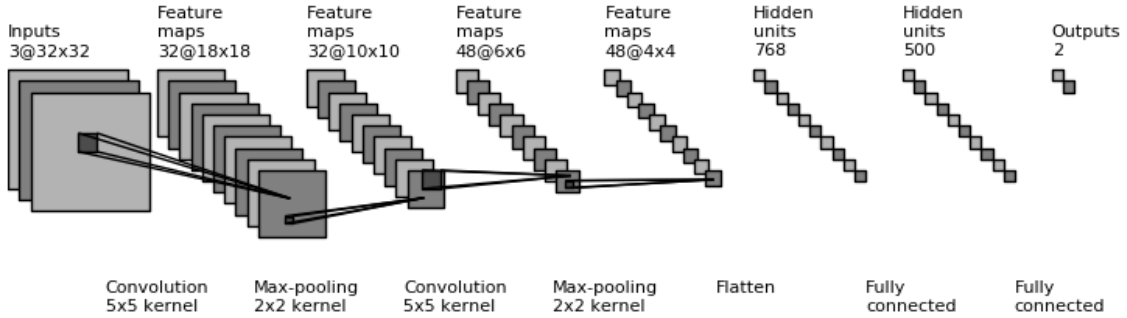
Convolutional layers on RPU array

Convolutional layer:

flatten each convolution kernel to a row of matrix

flatten image patches into a vector

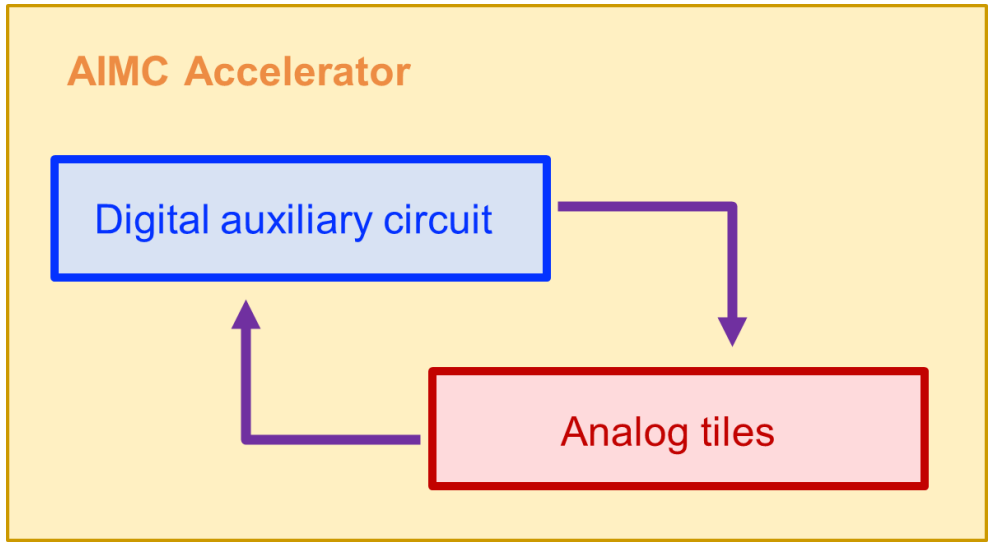


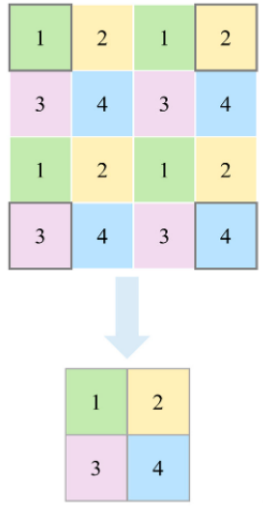


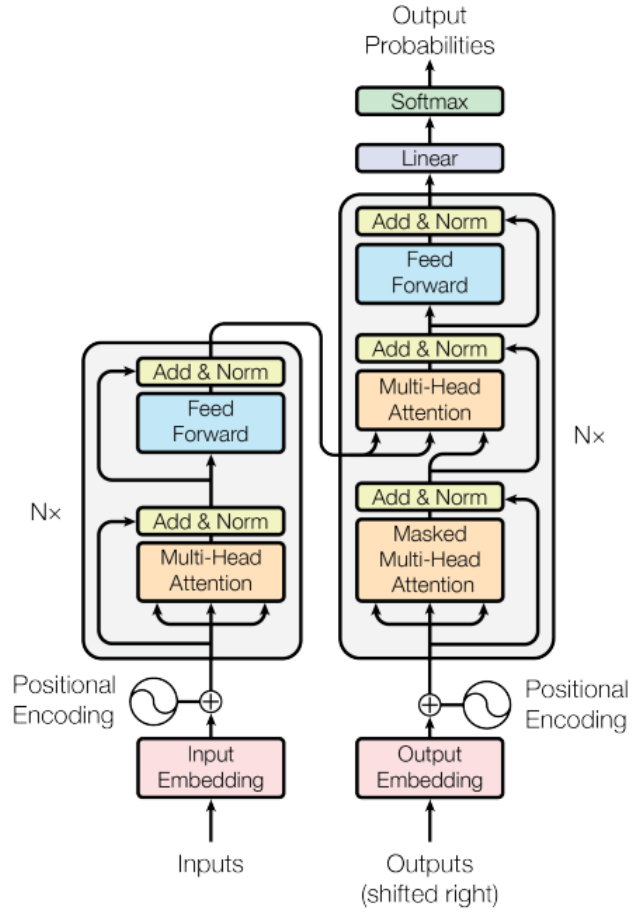
Non-linear operator on AIMC hardware

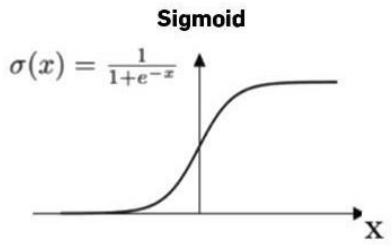
Non-linear operations are carried out in digital domain

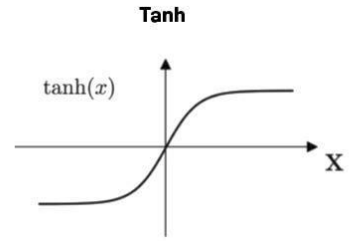
- Activation: Sigmoid, ReLU, LeakyReLU
- Down-sampling in CNN models
- Self-attention in Transform models

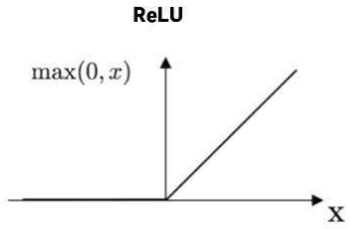




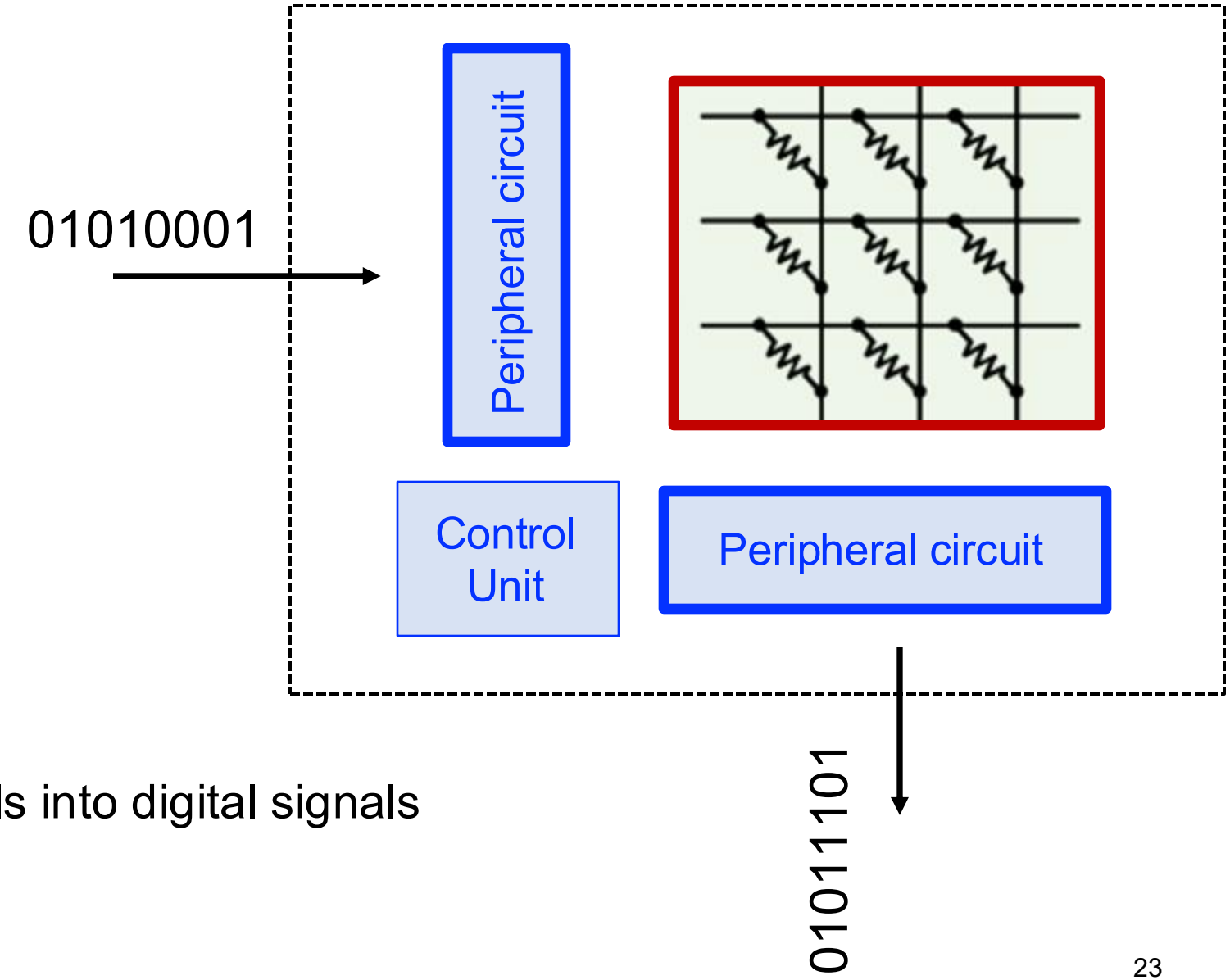








Analog-digital conversion



Peripheral circuit:

- Digital-Analog converter (DAC)
- Analog-Digital converter (ADC)

Quantize the current signals into digital signals

Weight programming

Pulse update: Send pulses to change conductance (thus weight)

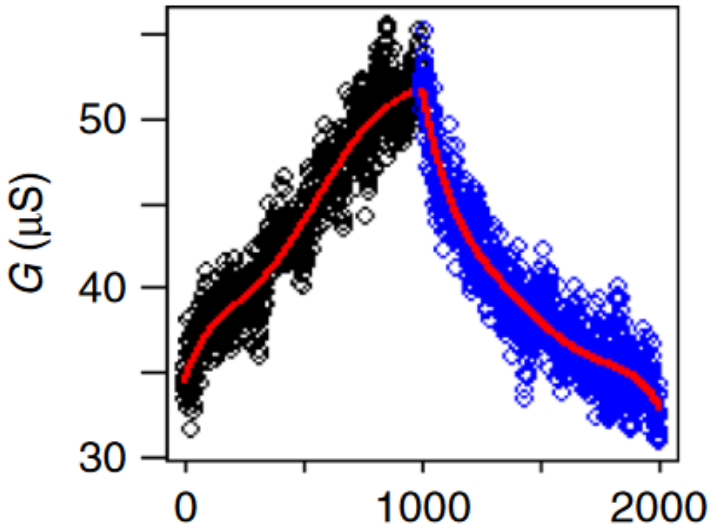
Program RPU for two purposes:

Inference: program pretrained model onto analog hardware

- Target weights are known

Training: keeping updating until model converge

- Target weights are unknown
- Adapt according to training objective



Different purposes leads to different update strategies

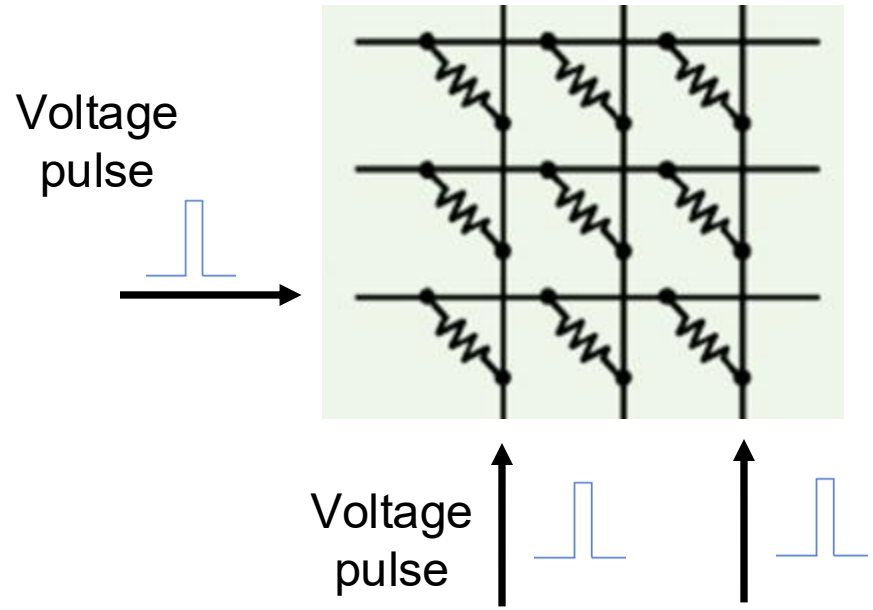
Weight programming for inference / training

Inference: expect to program the weights exactly

- Write-verification: ensure correct programming element-wise
- More accurate but circuits become more complicated

Training: only need to fulfill objective requirement

- Parallel update
- Idea: A pulse affects all elements simultaneously in the same row / column

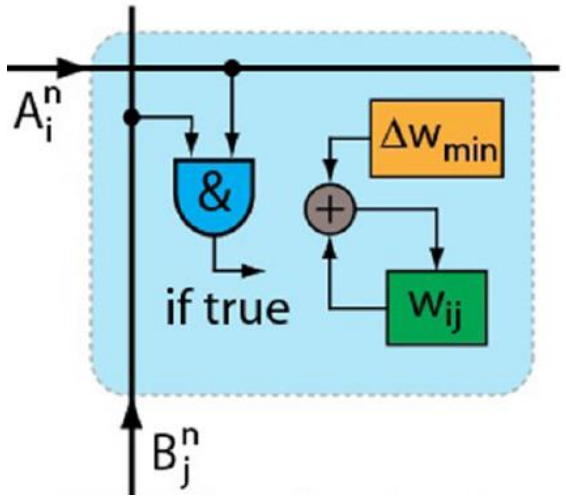


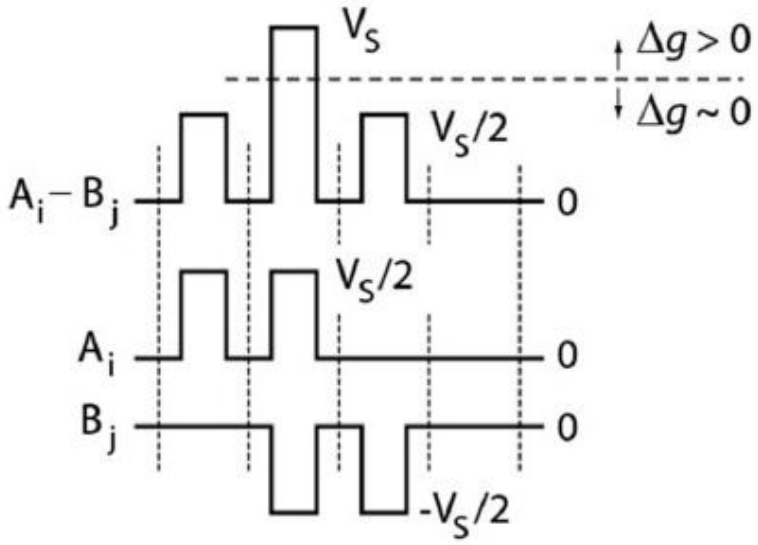
W Li, et al. "A 40-nm MLC-RRAM compute-in-memory macro with sparsity control, on-chip write-verify, and temperature-independent ADC references." **IEEE Journal of Solid-State Circuits 57.9 (2022): 2868-2877**.
Gokmen and Yurii. "Acceleration of deep neural network training with resistive cross-point devices: Design considerations." ***Frontiers in neuroscience*, 2016.**

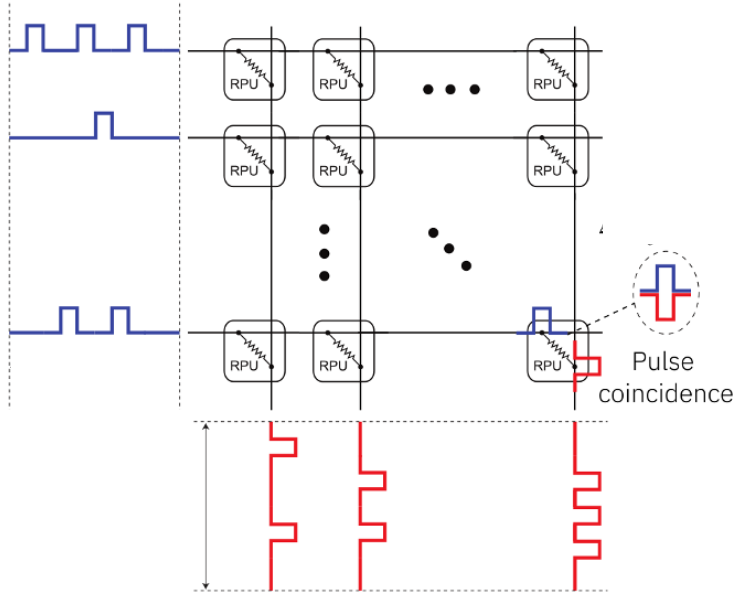
Parallel weight programming

Parallel update in crossbar tiles:

- Send pulses to both column & row
- Pulse coincident incurs weight update







Gokmen and Yurii. “Acceleration of deep neural network training with resistive cross-point devices: Design considerations.” ***Frontiers in neuroscience*, 2016.**

Weight copy

Direct copy:

Measure the conductance of each RPU

Parallel copy:

Taking MVM with one-hot vectors

3^{rd} *element*

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \cdot a_{11} + 0 \cdot a_{12} + 1 \cdot a_{13} + 0 \cdot a_{14} \\ 0 \cdot a_{21} + 0 \cdot a_{22} + 1 \cdot a_{23} + 0 \cdot a_{24} \\ 0 \cdot a_{31} + 0 \cdot a_{32} + 1 \cdot a_{33} + 0 \cdot a_{34} \\ 0 \cdot a_{41} + 0 \cdot a_{42} + 1 \cdot a_{43} + 0 \cdot a_{44} \end{bmatrix} = \begin{bmatrix} 1 \cdot a_{13} \\ 1 \cdot a_{23} \\ 1 \cdot a_{33} \\ 1 \cdot a_{43} \end{bmatrix} = \begin{bmatrix} a_{13} \\ a_{23} \\ a_{33} \\ a_{43} \end{bmatrix}$$

3^{rd} *column selected*

Parallel copy is more efficient if only a few columns are needed

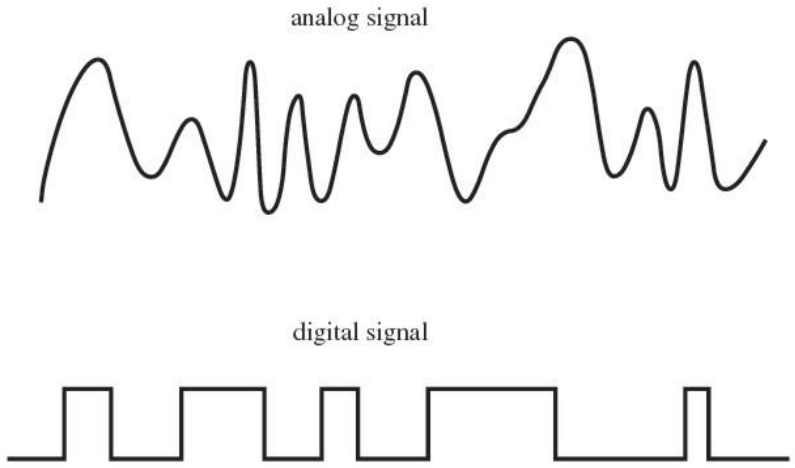
Takeaway

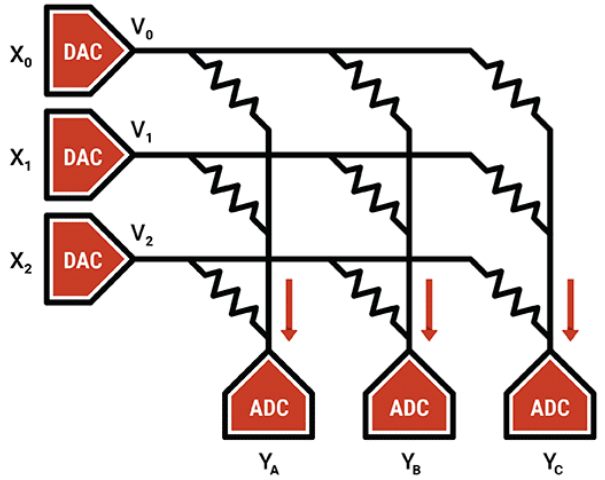
Basic operations on AIMC hardware:

- **Resistive processing unit (RPU)** leverages conductance to represent value
- **Matrix-vector multiplication (MVM):** Ohm's and Kirchhoff's laws
- Programming: **Pulse update**
  - Element-wise write & verification
  - Parallel programming
- Weight copy:
  - Measure conductance directly
  - Copying by MVM with one-hot vectors

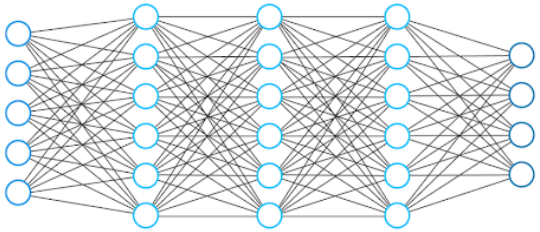
Outline

- Background of Analog In-memory Computing
- Analog Inference & Hardware-aware Retraining

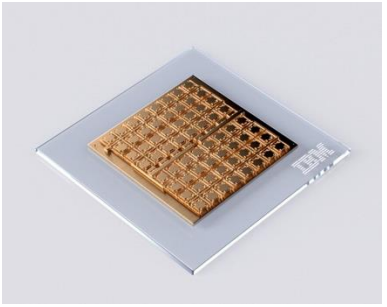




Inference on AIMC hardware



?

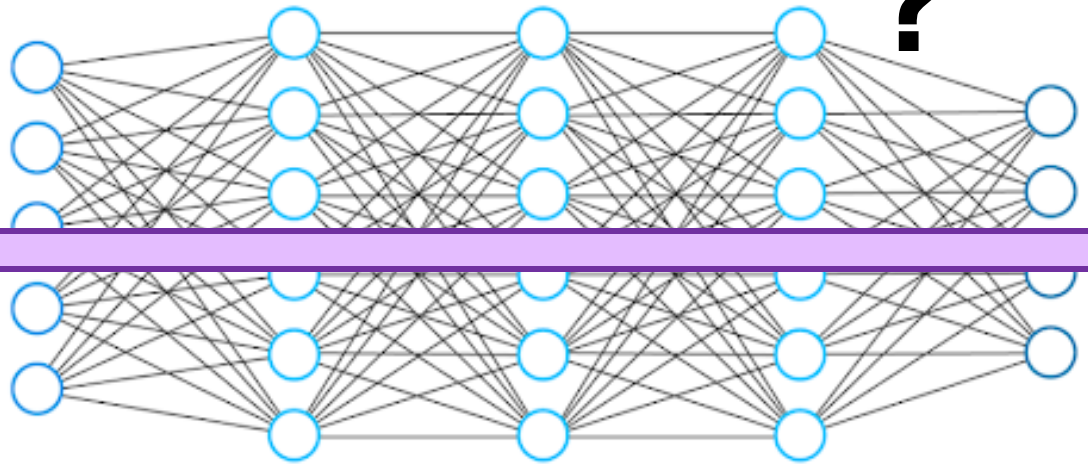


Pretrain a Model on GPUs

Pretrained Model

Program on AIMC hardware

?

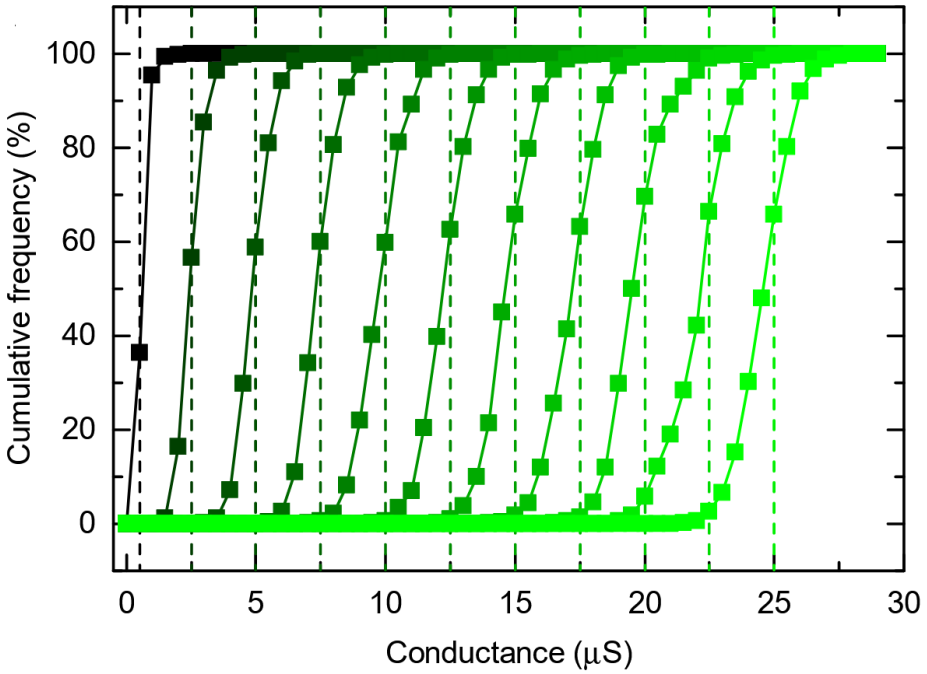


CAT

Both programming and inference can be problematic

Hardware imperfection I: Programming error

- The conductance values programmed on the hardware have a certain error
- Fig: Cumulative distributions of 11 representative iteratively programmed conductance levels

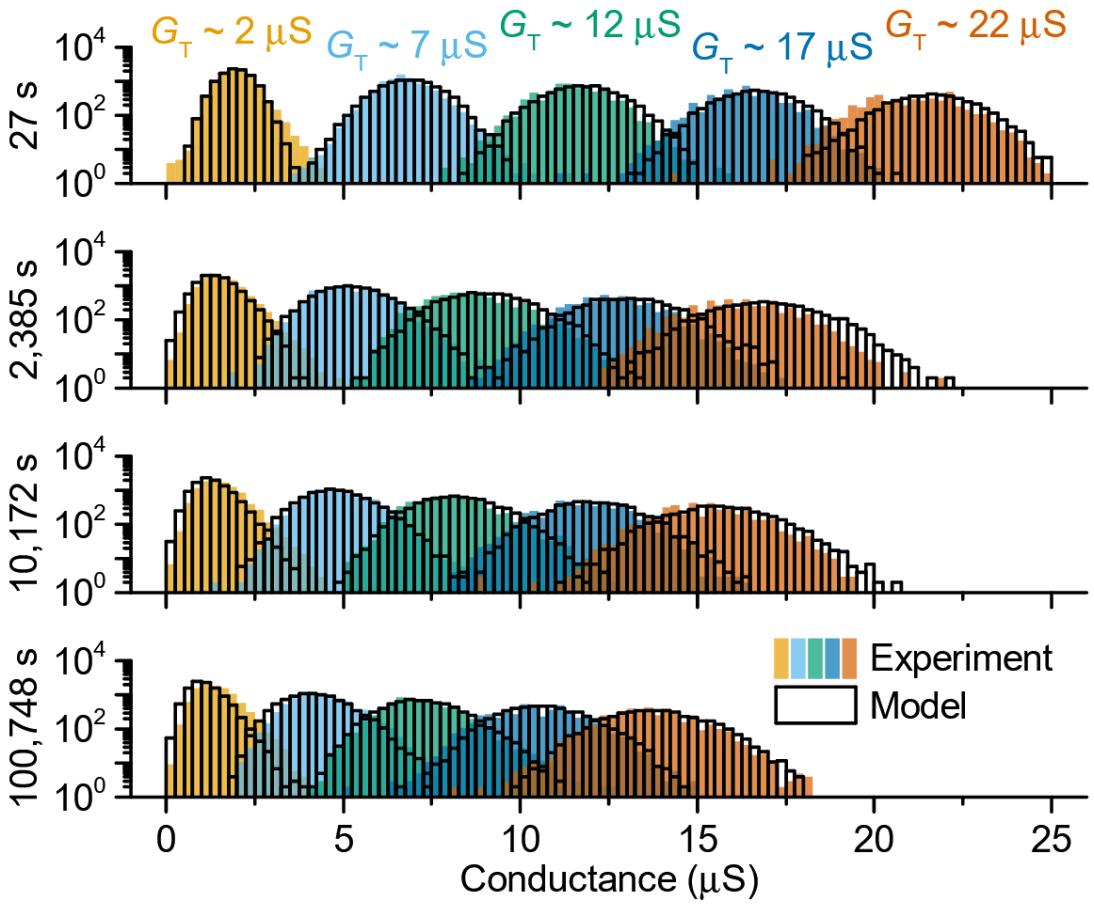


The programming processing is inaccurate

V Joshi, et al. "Accurate deep neural network inference using computational phase-change memory." **Nature communications 11.1 (2020): 2473.**

Hardware imperfection II: Weight drift

- Programmed weights drift over time
- PCM suffers from structural relaxation of the amorphous phase



V Joshi, et al. "Accurate deep neural network inference using computational phase-change memory." **Nature communications 11.1 (2020): 2473.**

Hardware imperfection II: Weight drift (con't)

Direct mapping	Test Error in %		
DNN	FP_{32}	1 hour	1 year
ResNet-32 CF10	5.80	11.68 0.16	15.21 0.40
WideRN-16 CF100	20.00	26.02 0.10	29.42 0.28
ResNet-18† ImNet	30.50	47.91 0.21	55.00 0.41
ResNet-50† ImNet	23.87	32.51 0.09	38.18 0.31
DenseNet-121† ImNet	25.57	47.38 0.24	59.81 0.71
WideRN-50† ImNet	21.53	40.07 0.13	45.45 0.30
BERT-base MRPC	14.60	15.99 0.23	18.36 0.30
Albert-base MRPC	15.08	31.50 0.10	31.62 0.03
Speech□ SWB300	14.05	16.11 0.02	16.32 0.02
LSTM PTB	72.90	73.16 0.01	73.28 0.01
RNN-T SWB300	11.80	28.16 0.10	29.08 0.16

Weight drift incurs significant accuracy degradation

V Joshi, et al. "Accurate deep neural network inference using computational phase-change memory." **Nature communications 11.1 (2020): 2473.**

Analog hardware-aware retraining

Idea: retrain with considering all the possible noise

min Loss(W)
over **Model weight (W)**

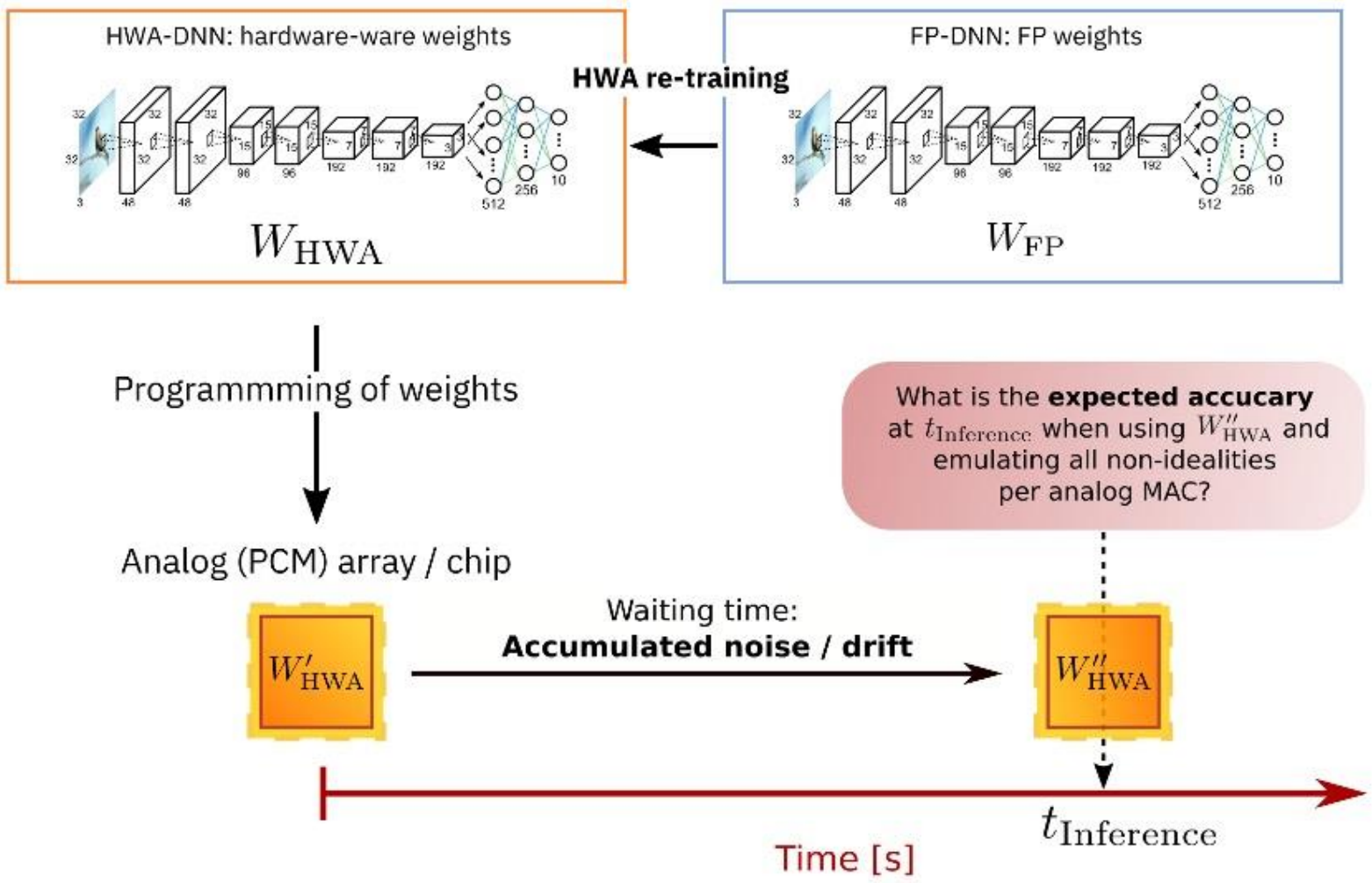
Original objective

min Loss(W + possible drift)
over Model Weight (W)

hardware-aware Retraining
(HWA) objective

V Joshi, et al. "Accurate deep neural network inference using computational phase-change memory." **Nature communications 11.1 (2020): 2473.**
MJ, Rasch, et al. "Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators." **Nature communications 14.1 (2023): 5282.**

Analog hardware-aware retraining (con't)



V Joshi, et al. "Accurate deep neural network inference using computational phase-change memory." **Nature communications 11.1 (2020): 2473.**
MJ, Rasch, et al. "Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators." **Nature communications 14.1 (2023): 5282.**

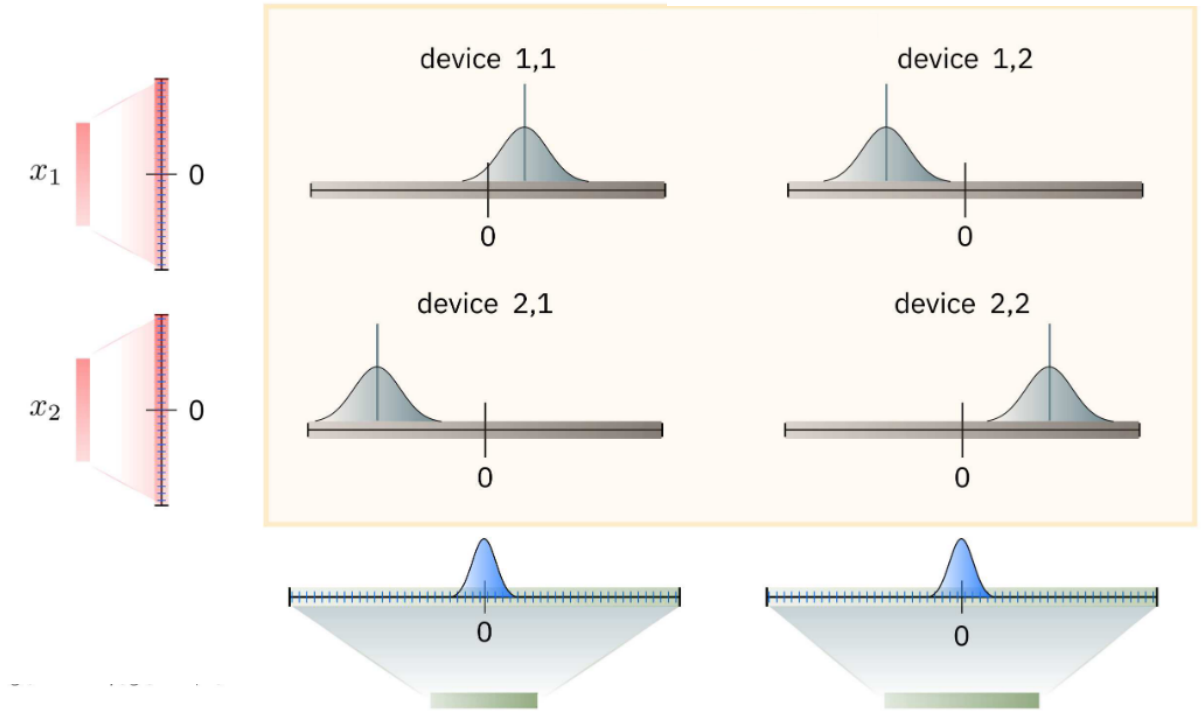
Hardware imperfection III: Quantization & read noise

- Weights suffers from instantaneous fluctuations due to the intrinsic noise

Output = Quad((Weight + noise) x Input + Bias)

Similar signals are quantized to same level

$$z = \mathrm{Quad}((W + \varepsilon)x)$$



MJ, Rasch, et al. "Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators." **Nature communications 14.1 (2023): 5282.**

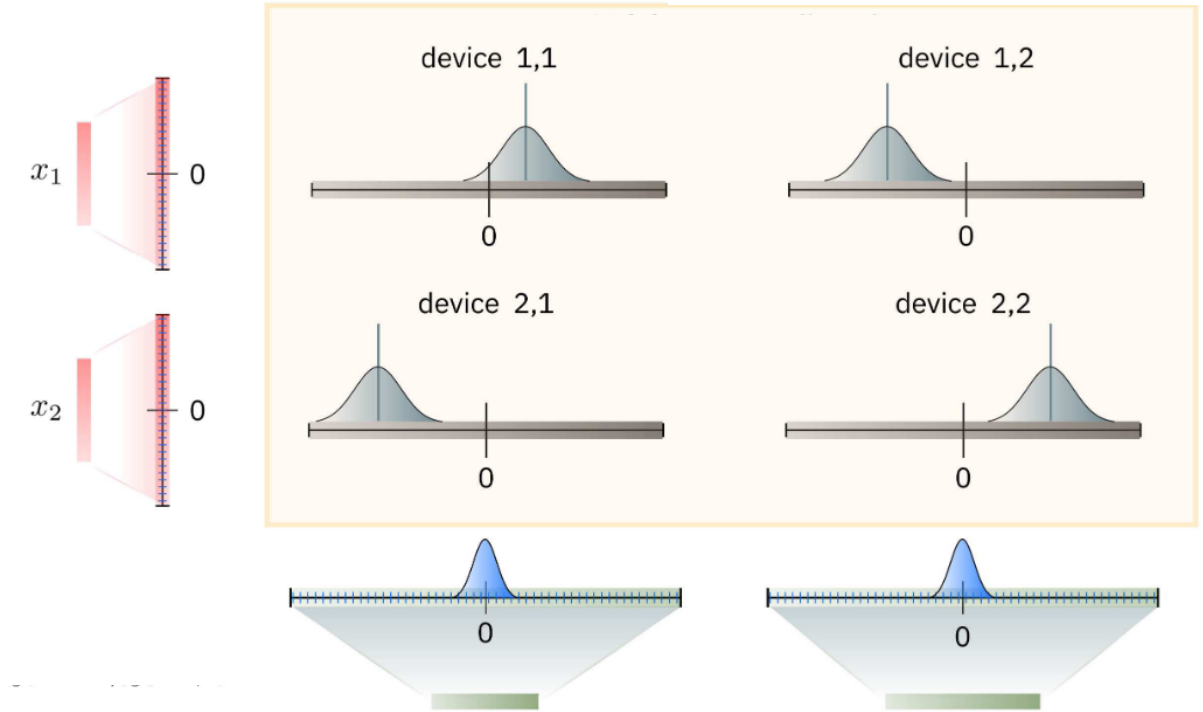
Hardware imperfection III: Quantization & read noise

- Solution: scale the input before MVM

$$\tilde{x} = \frac{x}{\alpha} \quad \tilde{z} = \mathrm{Quad}((W + \varepsilon)\tilde{x}) \quad z = \alpha\tilde{z}$$

- Equivalently

$$z = \alpha \cdot \mathrm{Quad}((W + \epsilon) \cdot \frac{x}{\alpha})$$



MJ, Rasch, et al. "Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators." **Nature communications 14.1 (2023): 5282.**